From Noisy to Usable: A Learned Skeleton Filter for Markerless Motion Capture

A temporal convolutional network trained on corrupted optical mocap, measured against the best low-pass filter on an actor it has never seen

Ravil Iskhakov
Independent technical animator

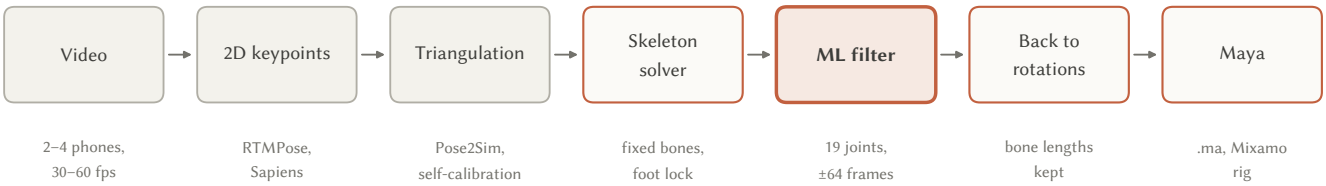


Figure 1: Pipeline of MoCap Studio. Orange outlines mark the parts built for this work, the filled box is the subject of this paper; 2D keypoints and triangulation use existing models and Pose2Sim. The filter works on the solved skeleton, so its output goes straight back into joint rotations and a Maya scene.

ABSTRACT

Markerless capture from a few phones gives a skeleton that jitters, drifts in depth, slides its feet and for a few frames swaps the left and right limbs. A low-pass filter removes the jitter but also cuts the fast motion that animators care about. We train a 608k-parameter temporal convolutional network that looks at ±64 frames (±2.1 s) of a 19-joint skeleton and predicts a correction for every joint. Training uses clean optical mocap (LAFAN1, actors 1–4) corrupted by a model of markerless errors: jitter, body and joint drift, foot sliding, short spikes and left/right swaps, followed by the pipeline's own 6 Hz smoothing. Training takes 3 minutes on one RTX 4080. On a held-out actor (13 takes, 89,853 frames) the mean joint error falls from 4.40 to 3.46 cm at medium noise and from 11.64 to 7.33 cm at strong noise, the acceleration error falls 2.2–3.2 times, left/right swaps nearly disappear, and hands and feet keep 98–99 % of their peak speed. The best-tuned Butterworth filter leaves the error almost unchanged (4.38 and 11.46 cm) and cuts foot peaks to 93 and 85 %. The cleaned positions are turned back into rotations with fixed bone lengths and exported to Maya. On two real phone takes the gain is smaller (jitter −10 to −13 %), and on a standing actor the filter unlocks feet that the solver had planted, which we report as the main open problem.

1 INTRODUCTION

Optical motion capture needs a studio, suits and markers; markerless capture needs only cameras. For an animator the price of markerless capture is noise. Our tool, MoCap Studio, films an actor with two to four phones, detects 2D keypoints, triangulates them with Pose2Sim [9], fits an anatomical skeleton with fixed bone lengths and writes a Maya scene (Figure 1). The solved skeleton is usable but not clean: the wrists shake, the pelvis drifts along the cameras' viewing direction, planted feet slide, and when one limb hides the other the 2D detector can swap left and right for a few frames.

The usual answer is a zero-phase Butterworth filter [13]. It works on jitter but has one knob, the cut-off: a low cut-off flattens strikes, jumps and foot contacts, a high one leaves the noise. It also cannot fix errors that are not high-frequency, such as a swapped limb or a slow drift. We wanted a filter that keeps fast motion, fixes structured errors, runs offline in seconds and can be trained without any ground truth from our own rig.

This paper describes such a filter and measures it honestly. Our contributions are:

- a noise model of markerless capture applied to clean optical mocap, so the network can be trained and tested against the truth;
- a small residual temporal network on a heading-aligned skeleton, trained with position, velocity and acceleration terms;
- a step that turns the cleaned positions back into joint rotations with fixed bone lengths, so the result stays a skeleton animators can edit;
- an evaluation against the best-tuned Butterworth filter on a held-out actor, and on two real phone takes without ground truth, including where the filter fails.

2 RELATED WORK

Denoising motion capture. Holden [3] trains a network on optical marker data corrupted with synthetic noise and solves clean joint transforms; we follow the same idea of corrupting clean mocap, but corrupt a solved skeleton the way markerless capture does. Holden et al. [4] learn a motion manifold with a convolutional autoencoder and use it, among other things, to remove noise. SmoothNet [15] refines per-frame pose estimates from video with a temporal network. **Temporal convolutions.** Dilated 1D convolutions see a long window at little cost [12]; VideoPose3D [10] uses them to lift 2D keypoint sequences to 3D. Our network is of this kind, applied after triangulation and solving. **Markerless capture.** Pose2Sim [9] triangulates 2D keypoints from calibrated cameras; RTMPose [5] and Sapiens [6] are the 2D models we use. Foot skate cleanup has a long history in animation [7]; Section 7 shows why it is still needed after our filter. **Data.** LAFAN1 [2] is a public set of optical mocap of five actors captured by Ubisoft La Forge; we use it as the clean source.

Table 1: Noise model. Jitter and joint drift are scaled per joint by 0.8 (hips, spine) to 1.5 (head, wrists, ankles, toes).

Error	Medium	Strong
Jitter, σ	1.2 cm	2.5 cm
Body drift, below 0.5 Hz	2 cm	5 cm
Joint drift, below 0.7 Hz	1 cm	2.5 cm
Foot sliding speed, below 1.5 Hz	6 cm/s	27 cm/s
Spikes (5–15 frames, 5–25 cm)	4 / min	12 / min
Left/right swaps (2–8 frames)	1 %	3 %
Final low-pass	6 Hz	6 Hz

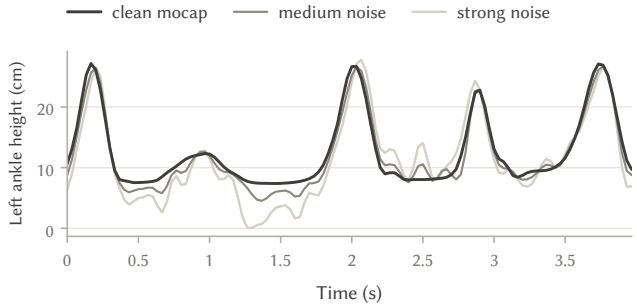


Figure 2: The two test levels on the left ankle of a walk from the test actor. Sliding, spikes and drift change the contact height; the final 6 Hz filter smooths the jitter.

3 TRAINING DATA AND NOISE MODEL

Clean data. LAFAN1 has 77 takes of five actors at 30 fps. We train on 53 takes of actors 1–4 (333,571 frames, 3.1 h), validate on every sixth take of the same actors (11 takes, 73,248 frames) and test on all 13 takes of actor 5 (89,853 frames), which the network never sees. Each frame is reduced to the 19 joints our solver produces: hips, two spine points, neck, head, shoulders, elbows, wrists, hip joints, knees, ankles and toes. The two spine points sit at 40 and 75 % of the path from the hips to the neck.

Random bodies. Every take is used eight times. One copy keeps the original body and no noise, so the network also learns to leave clean motion alone. The other seven get a new body, scaled by 0.85–1.25, with legs 0.93–1.07, spine 0.9–1.25, head 0.6–1.3, shoulders 0.75–1.3, upper arms 0.8–1.6, forearms 0.92–1.15 and hands 0.95–1.35 of the original, and spine points moved to 33–45 and 66–80 % of the hips–neck path.

Noise model. Table 1 lists the errors we add, all in metres on the joint positions, in this order: white jitter, larger at the head, wrists and feet; a slow drift of the whole body and of each joint; sliding of each foot as integrated coloured velocity; short bumps of 5–15 frames where a joint jumps by 5–25 cm, as when it is hidden; and swaps of the left and right arm or leg for 2–8 frames. Finally the sequence passes through the same zero-phase low-pass filter as our pipeline. Training copies use a random level, the strong level scaled by $t \sim U(0, 1.3)$, and a random cut-off of 4–9 Hz. Tests use two fixed levels, medium and strong, both with a 6 Hz cut-off (Figure 2).

4 NETWORK AND TRAINING

Input. The network sees the skeleton in a frame that turns with the actor. The heading is perpendicular to the horizontal line between the hip joints, smoothed at 2 Hz. Per frame the input has 58 numbers: the 18 joints relative to the hips in the heading frame, the hip velocity in the same frame and the hip height. The input

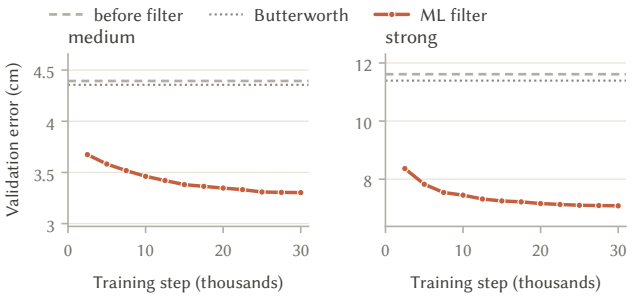


Figure 3: Validation error during training. Dashed: the noisy input; dotted: the best Butterworth filter. The curve is still falling at 30,000 steps.

is standardised with the training mean and deviation.

Network. A 1×1 convolution lifts the input to 128 channels. Six residual blocks follow, each a dilated convolution with kernel 5, GroupNorm [14] with eight groups, GELU, dropout 0.1 and a 1×1 convolution; the dilations are 1, 2, 4, 8, 16 and 1, which gives a window of ±64 frames. A last 1×1 convolution outputs a correction for each of the 19 joints in the heading frame, in units of 5 cm. Its weights start at zero, so the untrained network changes nothing. The network has 607,801 parameters.

Loss. With e_t the difference between the predicted and the true correction at frame t, the loss over a window is

$$\mathscr{L} = \overline{\|e_t\|^2} + 2\,\overline{\|e_t - e_{t-1}\|^2} + 4\,\overline{\|e_t - 2e_{t-1} + e_{t-2}\|^2}, \qquad (1)$$

so errors in position, velocity and acceleration are all penalised.

Training. We use AdamW [8] (learning rate 10^{-3}, weight decay 10^{-2}) with a one-cycle schedule [11] and 5 % warm-up, batches of 64 windows of 128 frames, gradient clipping at 1 and a left/right mirror of half the windows. Training runs 30,000 steps, 3.0 minutes on an RTX 4080. Every 2,500 steps the network is scored on the validation takes by the mean joint error plus 0.2 times the acceleration error; the best score came at the last step, and the validation error was still falling (Figure 3).

Inference. A take of 1,076 frames is filtered in 0.08 s on four CPU threads. Takes at other frame rates are resampled to 30 fps and back.

5 BACK TO JOINT ROTATIONS

Animators need rotations, not points. We run forward kinematics on the solved skeleton, filter the 19 joint positions and then rotate every bone by the shortest arc that points it at its cleaned child; the twist around the bone stays as the solver gave it. The pelvis and chest also use the cleaned hip and shoulder lines, and the hips take the cleaned position. Bone lengths do not change. As a check, using the unfiltered positions as targets changes rotations by less than 10^{-9} degrees and bone lengths by less than 10^{-7} mm. The result is written in the same skeleton format and as a Maya scene.

6 RESULTS ON A HELD-OUT ACTOR

Protocol. We compare the noisy input, the best Butterworth filter and our filter on all 13 test takes. The Butterworth cut-off is chosen on the validation takes with the same score as the network: 3 Hz for medium and 2 Hz for strong noise. Errors are measured against the clean mocap: mean joint error, mean error of acceleration, and peak speed as the 99th percentile of speed relative to the

Table 2: Held-out actor (LAFAN1 actor 5, 13 takes). Error: mean distance of the joints to the clean mocap. Acc.: mean error of acceleration. Peak: 99th percentile of speed relative to the clean mocap.

Noise	Method	Error (cm)	Torso, head	Arms	Legs	Acc. (mm/frame^2)	Peak wrists	Peak feet	Peak hips
Medium	Noisy input	4.40	3.74	4.41	4.80	9.76	127 %	105 %	101 %
	Butterworth, 3 Hz	4.38	3.69	4.37	4.82	5.23	100 %	93 %	98 %
	ML filter	**3.46**	**2.97**	**3.41**	**3.82**	**4.52**	98 %	99 %	99 %
Strong	Noisy input	11.64	9.19	10.80	13.80	19.78	177 %	127 %	106 %
	Butterworth, 2 Hz	11.46	9.05	10.59	13.62	6.38	93 %	85 %	96 %
	ML filter	**7.33**	**6.51**	**7.44**	**7.77**	**6.25**	98 %	98 %	99 %

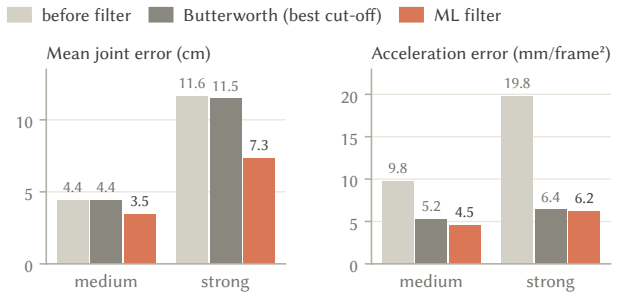


Figure 4: Mean joint error and acceleration error on the held-out actor.

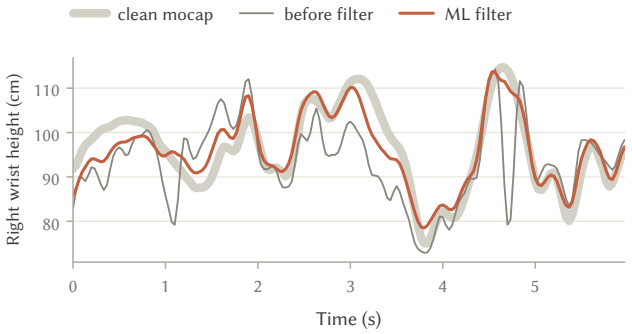


Figure 5: Height of the right wrist in a fight from the test actor, strong noise, 6 s.

clean take, where 100 % keeps the sharpness, less cuts it and more adds jerks.

Accuracy. Table 2 and Figure 4 summarise the test. The filter lowers the mean joint error by 21 % at medium and 37 % at strong noise; torso, arms and legs all improve. The Butterworth filter lowers it by less than 2 %: most of the error is drift, spikes and swaps, which a low-pass filter cannot remove.

Smoothness and sharpness. Both filters reduce the acceleration error, and at strong noise by about the same amount (6.38 and 6.25 mm/frame^2). The difference is what they keep: the Butterworth filter cuts the peak speed of the feet to 93 and 85 %, the network keeps 98–99 % for wrists, feet and hips. Figure 5 shows a fighting sequence: the network follows the strikes that the noise smeared.

Left/right swaps. We count frames where a wrist or ankle sits closer to the true position of the opposite one, among frames where the true left and right are more than 15 cm apart. Such frames fall from 1.07 to 0.02 % at medium and from 5.63 to 0.37 % at strong noise.

Consistency. Every one of the 13 test takes improves, from walking and running to fighting, dancing and getting up from the floor (Figure 6). The filtered error ranges from 3.29 to 3.74 cm at medium and from 6.92 to 7.92 cm at strong noise.

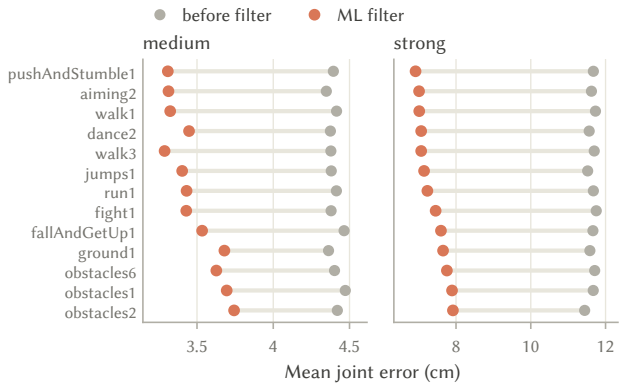


Figure 6: Mean joint error per test take before and after the filter. No take gets worse.

Table 3: Real takes without ground truth. Jitter: mean second difference of joint positions (mm/frame^2), before → after the filter.

	Classroom	orest_2
Jitter, all joints	3.34 → 3.00	1.56 → 1.36
torso, head	2.20 → 1.85	1.13 → 0.86
arms	4.08 → 3.71	2.99 → 2.56
legs	3.49 → 3.20	0.75 → 0.78
Peak speed, wrists (m/s)	5.08 → 4.72	1.72 → 1.52
Peak speed, feet (m/s)	4.10 → 3.70	1.05 → 1.22
Mean joint shift (cm)	2.81	2.40

7 REAL PHONE TAKES

Real takes have no ground truth, so we measure jitter as the mean magnitude of the second difference of joint positions, peak speed as the 99th percentile of speed, and how far the filter moves the joints. We use two takes processed by MoCap Studio with solver B: the public FreeMoCap [1] "classroom" recording with Sapiens2-1B keypoints (1,076 frames), and our own take *orest_2*: two phones 34° apart, RTMPose-x keypoints, an actor standing still and spinning a staff (871 frames).

What improves. On both takes the torso and arms get calmer (Table 3): torso jitter falls by 16 and 24 %, arm jitter by 9 and 14 %. The overall gain, 10–13 %, is much smaller than on the test set: the solver's own priors and smoothing already removed much of the jitter, and real errors are not exactly our noise model. The peak wrist speed falls by 7–12 %; without ground truth we cannot tell which part is removed noise and which is cut motion.

What breaks: planted feet. In *orest_2* the actor stands still and the solver's foot lock holds the ankles within 0.2 cm for 28 seconds. After the filter the ankles wander up to 5 cm from their place, move at 7 cm/s on average instead of 0.15, and leg jitter on

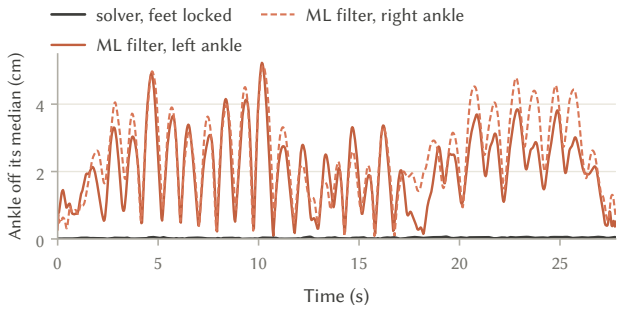


Figure 7: *orest_2*: horizontal distance of the ankles from their median position while the actor stands still. The solver keeps the feet planted; the filter makes them drift.

the standing part grows by 35 % (Figure 7). The network has no notion of contact: it treats a perfectly still foot like any other joint and adds its small corrections. The fix is to lock the feet again after the filter; MoCap Studio already has contact detection and leg IK for its Maya export, but the two steps are not yet chained.

8 LIMITATIONS AND FUTURE WORK

The noise model is hand-made from what we see in our captures; it is not measured against ground truth from our rig, and the gap between the test set and the real takes shows it is incomplete. LAFAN1 is one studio and one capture style; fingers are not covered. The filter knows nothing about foot contact, which is its main practical weakness. Training was short and the validation error was still falling. Next we plan to add contact as an input and output, to train longer, and to record our own takes together with ground truth: with pressure insoles for contact and, where possible, with optical mocap.

9 CONCLUSION

A small temporal network, trained in three minutes on clean optical mocap with a model of markerless errors, removes a third of the error that strong noise adds and most left/right swaps, and keeps fast motion that a low-pass filter cuts. Its output returns to joint rotations and to Maya without changing the skeleton. On real phone takes it calms the torso and arms but must be followed by foot locking.

ACKNOWLEDGMENTS

LAFAN1 is provided by Ubisoft La Forge under CC BY-NC-ND 4.0. The “classroom” recording is a FreeMoCap sample. The code was written with the help of an AI assistant (Claude by Anthropic).

REFERENCES

[1] FreeMoCap Foundation. FreeMoCap: free and open-source markerless motion capture. https://github.com/freemocap/freemocap.

[2] Félix G. Harvey, Mike Yurick, Derek Nowrouzezahrai and Christopher Pal. 2020. Robust Motion In-betweening. *ACM Transactions on Graphics* 39, 4.

[3] Daniel Holden. 2018. Robust Solving of Optical Motion Capture Data by Denoising. *ACM Transactions on Graphics* 37, 4.

[4] Daniel Holden, Jun Saito and Taku Komura. 2016. A Deep Learning Framework for Character Motion Synthesis and Editing. *ACM Transactions on Graphics* 35, 4.

[5] Tao Jiang, Peng Lu, Li Zhang, Ningsheng Ma, Rui Han, Chengqi Lyu, Yining Li and Kai Chen. 2023. RTMPose: Real-Time Multi-Person Pose Estimation based on MMPose. arXiv:2303.07399.

[6] Rawal Khirodkar, Timur Bagautdinov, Julieta Martinez, Su Zhaoen, Austin James, Peter Selednik, Stuart Anderson and Shunsuke Saito. 2024. Sapiens: Foundation for Human Vision Models. In *ECCV*.

[7] Lucas Kovar, John Schreiner and Michael Gleicher. 2002. Footskate Cleanup for Motion Capture Editing. In *Symposium on Computer Animation*.

[8] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *ICLR*.

[9] David Pagnon, Mathieu Domalain and Lionel Reveret. 2022. Pose2Sim: An open-source Python package for multiview markerless kinematics. *Journal of Open Source Software* 7, 77.

[10] Dario Pavllo, Christoph Feichtenhofer, David Grangier and Michael Auli. 2019. 3D Human Pose Estimation in Video with Temporal Convolutions and Semi-supervised Training. In *CVPR*.

[11] Leslie N. Smith and Nicholay Topin. 2019. Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates. arXiv:1708.07120.

[12] Aäron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior and Koray Kavukcuoglu. 2016. WaveNet: A Generative Model for Raw Audio. arXiv:1609.03499.

[13] David A. Winter. 2009. *Biomechanics and Motor Control of Human Movement* (4th ed.). Wiley.

[14] Yuxin Wu and Kaiming He. 2018. Group Normalization. In *ECCV*.

[15] Ailing Zeng, Lei Yang, Xuan Ju, Jiefeng Li, Jianyi Wang and Qiang Xu. 2022. SmoothNet: A Plug-and-Play Network for Refining Human Poses in Videos. In *ECCV*.